

Learning Management Portal — Build & Status Report

A complete record of what was redesigned, fixed, and built; what requires a backend server; and where to deploy the project for free.

Build: passing (38 routes)

Branch: feat/mtu-redesign-and-logic-fixes

Stack: Next.js 16 · React · Zustand · Tailwind

Architecture: client-only (localStorage)

Executive summary

The portal was taken from a rough AI-generated draft to a coherent, institution-grade product: a full visual redesign, a 70-point logic audit with fixes, a proper academic-calendar engine, and four new feature modules — all committed and building green. Because the app is intentionally client-only (no backend/database; data persists in the browser's localStorage), some specifications can only be simulated until a server is added; those are listed honestly in their own section.

Foundation work (before the feature phases)

- **Full UI redesign** to a Modern Institutional standard in MTU's real colours (forest green + navy), with proper typography and AI-slop purged (emoji, gradients, glassmorphism, glow effects) across ~40 files.
- **70-finding logic audit, fixed:** suspended-login gate, seed-data ID mismatches, per-recipient message scoping, dead/looping routes, duplicate live-class participants, admin self-lockout guard, and more.
- **Enrollment coherence:** one single source of truth for "a student's courses," used consistently everywhere.
- **Academic-calendar engine:** open/close **semester registration lock**, level-gated + semester-gated course registration, and promotion fully decoupled from sessions (per-student level editor + a deliberate bulk-promote tool).
- **Other:** dynamic Live-class page, working Mistral AI chatbot/advisor, sliding login image carousel, dropdown & chatbot fixes.

The four feature phases

PHASE	MODULE	STATUS	COMMIT
1	Notifications & Notepad	Shipped	5e6a7bd
2	Quiz module	Shipped	c645359
3	Library & Past Questions	Shipped	a67f48a
4	Assignments · Email · Video	Shipped	0c8a0b5

Phase 1 — Notification system + In-portal Notepad

- **Notification bell** (student, lecturer, admin): unread badge, dropdown with type icons + relative times, click-through links, mark-all-read, clear; full **history page**; per-type **preferences** in settings.
- **Per-user notifications**: targeted by recipient/role/type via a single `pushNotification()` helper that every other module uses.
- **Floating notepad**: multiple notes, **auto-save** while typing, search, pin, course tagging, word count, print/export — private per user, persists across sessions.

Phase 2 — Full Quiz module

- **Lecturer builder**: MCQ / True-False / Short-answer questions with marks, shuffle questions & options, one-at-a-time or all-at-once display, time limit, attempts, draft/publish; edit locks to deadline-only once a student has started.
- **Student take-flow**: status tags (Upcoming/Active/Closed), always-visible countdown with **auto-submit**, flag-for-review, **30-second autosave**, **tab-switch anti-cheat** (3 switches = auto-submit + lecturer alerted), copy-blocking, instant MCQ/TF score, full result breakdown.
- **Grading & dashboards**: auto-grade MCQ/TF, manual short-answer grading (student notified), submissions dashboard (scores, time taken, flags), CSV + print export; **admin** oversight with system analytics.

Phase 3 — Digital Library + Past Questions

- **Library**: lecturers upload materials (link or small file) tagged by week/topic and type, with visible/hidden control; students browse their registered courses' materials, preview/download, search, and see a "New" badge; admin storage oversight.
- **Past Questions**: archive by year / semester / exam type, **cross-department** browsing, optional answer-scheme with visibility control; admin bulk management.

Phase 4 — Assignments + Email + Video

- **Assignments**: lecturer builder (formats, reference doc, rubric, per-student extensions), **hard deadline-lock**, resubmission-replaces-previous, grading + feedback, submissions dashboard + CSV/print report; student submit flow with PDF/DOCX + size validation and live countdown.
- **Email notifications**: styled HTML **welcome** and **login-alert** emails, the login alert including **IP geolocation** (city/state/country), device/browser, and time, plus a "Secure my account" action; fired automatically on signup/login.
- **Real video classes**: classrooms now embed **Jitsi Meet** (genuine camera/mic/screen-share, free, no API key) with **attendance tracking** (join/leave, present/late) and a hard end-time lock; lecturer scheduling + attendance report.

What needs a real server (current honest limitations)

The app deliberately has no backend — it runs entirely in the browser with `localStorage` as its "database." These items are stubbed, simulated, or capped until a server + database is added:

CAPABILITY	NOW	WHY IT NEEDS A SERVER
Email delivery to real inboxes	Preview only	Emails are fully built and viewable via Ethereum preview links; delivering to real addresses needs valid SMTP credentials (Gmail/SendGrid/Mailgun) in environment variables.
File uploads > ~1 MB	Capped	localStorage holds only ~5 MB total; real files need object storage (S3, Cloudinary, Supabase Storage).
Shared / multi-user data	Per-browser	Data lives in each browser. A lecturer seeing a student's submission only works in the same browser. A real database makes it shared across users/devices.
Real-time updates & offline queue	Local	True live sync between users needs WebSockets / a service (Pusher, Supabase Realtime).
Mobile push notifications	Not built	Requires a mobile app / service worker + a push service (FCM).
Tamper-proof quizzes & secure grading	Client-side	Answer keys are in the browser; trustworthy grading must run on a server the student can't inspect.
IP geolocation accuracy	Deploy-only	On localhost the IP is loopback ("Location unavailable"); it resolves real cities once hosted on a public server.

Good news: because the app is client-only, it **deploys and runs as-is for free** with no database to set up — perfect for a demo. Adding a backend later is an upgrade path, not a prerequisite.

Deploying for free (hosting options)

This is a Next.js app with server API routes (for email), so it needs a **Node.js host** — not a static-only host. Set two environment variables on whichever you pick: `MISTRAL_API_KEY` (AI chatbot) and your SMTP creds (e.g. `EMAIL_USER` / `EMAIL_PASS`) if you want real email.

★ RECOMMENDED

Vercel

Built by the makers of Next.js — zero-config, generous free tier, API routes & AI route work out of the box. Connect the GitHub repo, set env vars, deploy. Best fit by far.

YOU MENTIONED THIS

Render

Free "Web Service" (Node). Build: `npm install && npm run build`, Start: `npm start`. Works well; the free tier sleeps after ~15 min idle (a few-second cold start on first hit).

ALTERNATIVE

Netlify

Free tier with Next.js support via its adapter. Connect repo, set env vars. Solid for front-end-heavy apps.

ALTERNATIVE

Railway

Very simple Node deploys, GitHub integration, small free usage allowance. Good for quick demos.

ALTERNATIVE

Koyeb / Fly.io

Free Node hosting tiers with global edge. A little more setup, but capable and free for light usage.

IF YOU ADD A BACKEND

Supabase / Firebase

Free hosted Postgres/Mongo-style database + auth + file storage + realtime — the easiest way to lift the “needs a server” limitations later.

Fastest path: push to GitHub (already done) → import the repo on **Vercel** → add the two env vars → Deploy. You’ll have a public link in a couple of minutes, and the per-browser localStorage model means it just works with no database.

If you later go production-grade

- **Database:** Supabase / Postgres (or MongoDB) to make data shared and permanent.
- **File storage:** Cloudinary / S3 / Supabase Storage for real material, past-question and assignment uploads.
- **Server-side quiz grading & auth:** move the answer key and grading behind the API; add hashed passwords + sessions.
- **Real email & realtime:** SendGrid/Mailgun for delivery; Pusher/Supabase Realtime for live notifications.